

AI 생성 음악 탐지와 생성기 식별

Clean 성능 · 미지 생성기 · MP3 재압축 · Generator Fingerprint

4

탐지 모델

12

AI 음악 생성기

296

기준 원곡

기계학습과 딥러닝 4조.

오승재 · 이의진 · 서승아 · 양지민 · 김재현

0. AI 음악은 이미 '특수한 콘텐츠'가 아니다

VIRAL CASE · GG EZ

먼저 유행하고, AI 활용은 뒤늦게 주목됐다

- 인스타그램 릴스·틱톡의 아이돌 챌린지로 확산
- 곡 자체가 먼저 소비된 뒤 제작 과정의 AI 활용이 화제

제작자가 가사·스타일을 직접 설계하고 AI를 활용한 **인간-AI 협업 사례**

01 SHORT-FORM

Instagram Reels · TikTok

02 VIRAL

아이돌·크리에이터 챌린지

03 AI REVEAL

AI 활용 사실이 뒤늦게 주목

AI 여부는 더 이상 청취만으로 자명하지 않다

0. 논쟁은 저작권에서 '확인 가능성'으로 이어진다

01 학습 데이터

무단 학습인가,
공정 이용인가?

2024 · 주요 음반사 vs Suno·Udio

02 저작자성

AI 산출물의
저작자는 누구인가?

2025 · 인간의 창작적 기여가 보호 기준

03 라이선스

금지에서
허가·보상 구조로

2025 · 일부 소송은 합의·제휴로 전환

그래서, 유통되는 음악이 AI 생성인지 어떻게 확인할 것인가?

탐지

강건성

생성기 식별

FMA 인간 음악 ↔ Echoes의 Text-to-Audio 완전 생성 음악

1. 데이터 출처: Echoes와 FMA

FAKE · AI 생성 음악

Echoes

음악 딥페이크 탐지용 공개 데이터셋
원곡과 의미적으로 유사한 AI 음악을 구성

연구진 Pascu 외 (2026)

공식 배포처 · Hugging Face
huggingface.co/datasets/Octavian97/Echoes

사용 범위

TTA만 선택 → 정제 후 FAKE 3,162트랙
ATA 제외 · 12개 생성기 · 3개 장르

REAL · 실제 음악

FMA (Free Music Archive)

음악 분석 연구용 공개 데이터셋
원자료 전체 106,574트랙 (공식 소개 기준)

연구진 Defferrard 외 (ISMIR 2017)

공식 배포 안내 · GitHub 저장소
github.com/mdeff/fma

사용 범위

기준 원곡과 매칭한 REAL 296트랙
Echoes의 original_audio ↔ FMA 원곡

TTA = Text-to-Audio / ATA = Audio-to-Audio · TTA는 동일 멜로디의 복제를 의미하지 않음

1. Echoes - FMA 매핑 및 데이터 확보

매칭 키 만들기

ECHOES original_audio ↔ FMA title+artist를 정규화 후 비교

A Dark Blue Arc - Pipe Choir

→ **a dark blue arc - pipe choir**

① 유니코드 NFKC ② 소문자 ③ 연속 공백 → 1칸

매칭 결과

279

정확히 1개 후보

17

복수 후보

0

무매칭

296개 원곡 전부 최소 1개 후보 확보

최종 REAL = 296곡
(original_audio 296 → FMA track_id 296, 1:1 매핑)

복수 후보 처리 규칙

- ① 허용 라이선스 후보 우선
- ② 장르 일치 후보 우선
- ③ 동물이면 최저 track_id 선택

실제 사례

I'm Gonna Try To Reach — Los Lllamarada

track_id 28763

98초 · NoDerivatives

track_id 84389

98초 · NonCommercial·ShareAlike

규칙 적용 시 *fallback* → *genre* 동률 → *track_id*로 결정

최종 선택 → track_id 28763

1. 원곡 그룹 분할과 데이터 누수

파일 단위 랜덤 분할

같은 원곡에서 파생된 파일끼리 서로 공유 가능

TRAIN

REAL A

FAKE B

TEST

FAKE A

REAL B

원곡 A가 Train과 Test에 동시에 노출

모델이 AI 생성 흔적보다 원곡의 멜로디와 구조를 기억할 위험

원곡 그룹 단위 분할

원곡을 먼저 분할하고 파생 데이터를 함께 배정

SOURCE

원곡 A

하나의 원곡 그룹

모두 TRAIN에 배정

REAL A

Suno A

Udio A

MusicGen A

296개 원곡을 70:15:15로 분할

장르 기준 층화 추출 · random seed 42

같은 원곡에서 파생된 모든 REAL·FAKE Track과 Segment는 동일한 split을 상속한다

1. 데이터 그룹 분할

Source

기준 원곡 A

Track

REAL A · 생성기별 FAKE A

Segment

각 Track의 10초 입력

분할 원칙

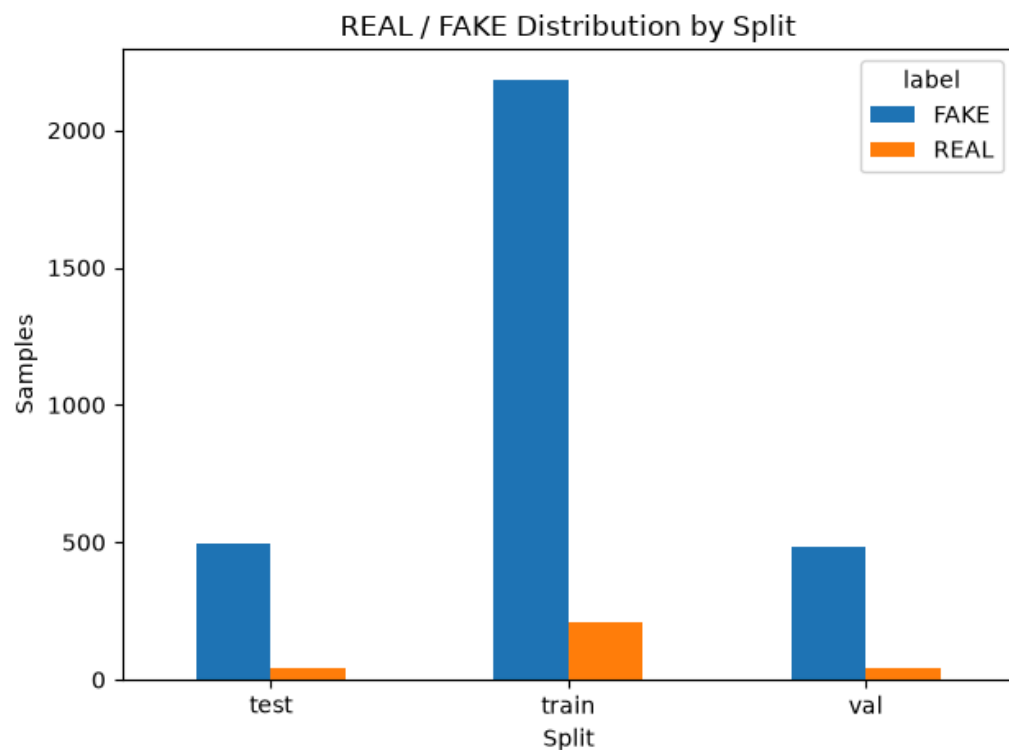
원곡을 먼저 분할하고
관련 Track과 Segment에 상속

같은 원곡 계열이 Train과 Test에
동시에 들어가는 상황 방지

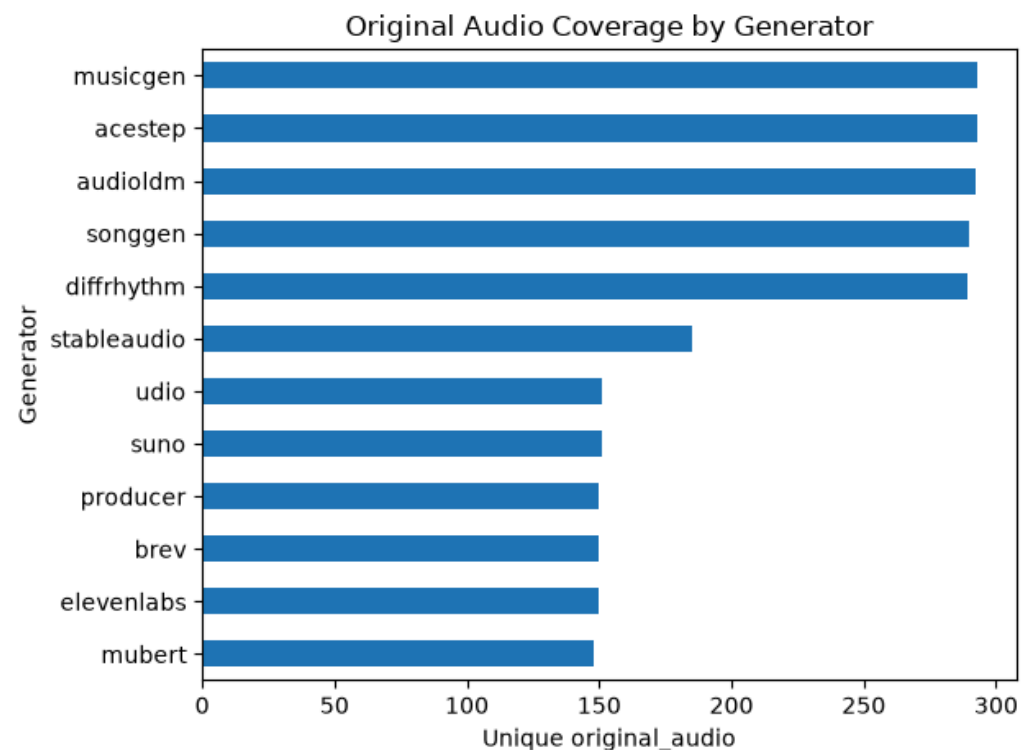
구분	원곡	전체 Track
Train	207	2,392
Validation	44	527
Test	45	539
합계	296	3,458

원곡 중복 0개 · 장르를 고려한 70:15:15 분할 · seed 42

1. EDA : 클래스 불균형과 생성기 구성



실제 Notebook 출력 · split별 REAL / FAKE



실제 Notebook 출력 · 생성기별 원곡 범위

FAKE 91.44% · 생성기별 원곡 범위 차이 → 균형 지표와 후속 통제 실험 필요

1. EDA : 클래스 불균형과 생성기 구성

Class imbalance

91.44%

FAKE · 3,162 tracks

8.56%

REAL · 296 tracks

FAKE 데이터가 압도적으로 많은 클래스 불균형 존재
→ Accuracy 단독 사용 위험

Generator coverage

149~300

생성기별 track 수

50~99%

source coverage 범위

생성기마다 사용한 원곡 범위가 달라 원곡을 단서로 외
울 위험

Genre distribution

Electronic

23~40%

Pop

22~34%

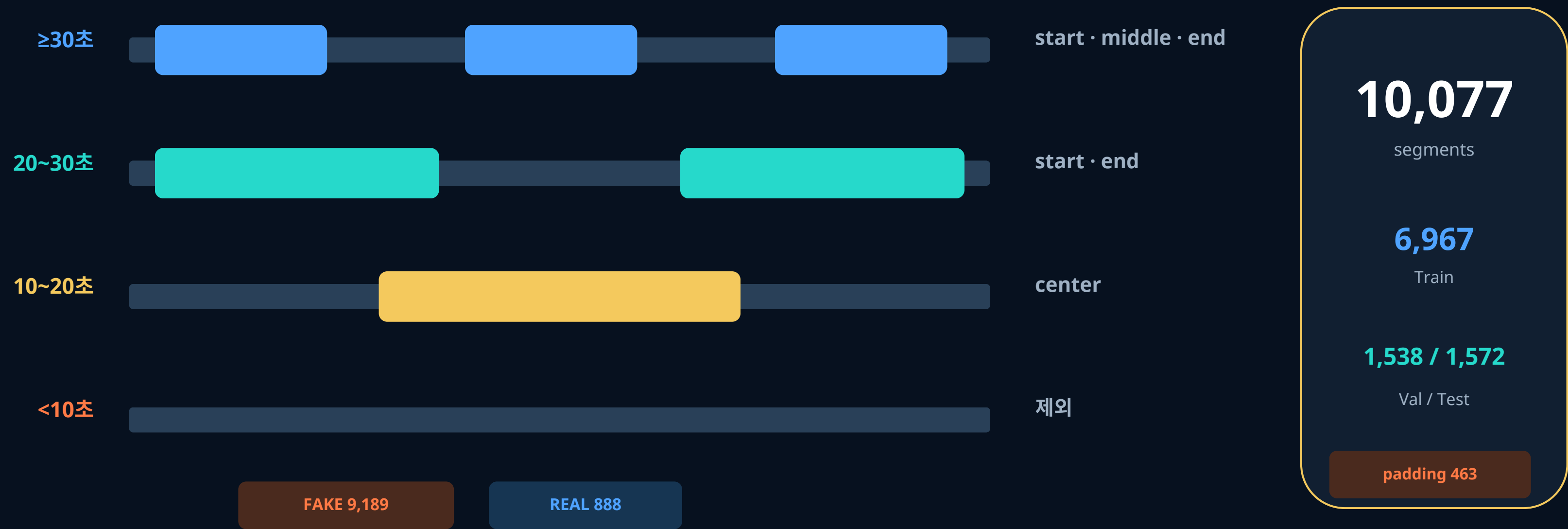
Rock

33~44%

위험: AI 생성 특징이 아니라 특정 장르를 단서로 예측
할 가능성

FAKE 91.44% · 생성기별 원곡 범위 차이 → 균형 지표와 후속 통제 실험 필요

1. 10초 Segement



주의 구간은 서로 겹치지 않지만, 같은 Track에서 나온 segment는 완전히 독립적인 표본이 아니다. 짧은 decode 오차는 zero-padding.

1. 평가 기준

데이터 사용

- **Train:** 모델·표준화 학습
- **Validation:** 판정 기준·epoch·level 선택
- **Test:** 고정된 설정으로 최종 평가

Track score = 같은 곡에 속한 Segment score의 평균

지표	해석
ROC-AUC ↑	REAL보다 FAKE에 높은 점수를 주는 능력
EER ↓	오탐률과 누락률이 가까운 지점의 오류
Balanced Accuracy ↑	REAL·FAKE 재현율의 평균
Macro-F1 ↑	클래스별 F1을 같은 비중으로 평균

Test EER은 ROC 요약 지표 · 실제 Test 판정에는 Validation에서 정한 threshold 사용

2. 모델 비교 설계

WHY · 전체 설계

모델	입력	선택 이유	학습 범위
Logistic Regression	266-D 음향 특징	선형 기준선 확보	선형 가중치
RBF-SVM	동일한 266-D	비선형 분리 효과 확인	서포트 벡터 기반 경계
Log-Mel CNN	시간-주파수 지도	국소 패턴 직접 학습	CNN 전체
MERT + LR	level별 768-D	음악 사전학습 표현 검증	LR만 학습

모든 모델의 원곡 분할 동일 · 각 모델의 Validation 선택 기준 고정

2-1. Logistic Regression

WHY · HOW

모델 소개

여러 특징의 **가중합**으로
두 클래스를 구분하는 선형 모델

Why? : 복잡한 모델의 개선 폭을 비교할 기준선

예측 과정

266개 특징에 서로 다른 가중치 적용

optimizer : $z = w_1x_1 + \dots + w_{266}x_{266} + b$

sigmoid 함수 : $\text{FAKE score} = 1 / (1 + \exp(-z))$

266-D 특징

구간별 음향 통계

StandardScaler

Train 통계로 표준화

LR + sigmoid

0~1 FAKE 점수

출력 점수를 곡별로 평균한 뒤 Validation threshold와 비교하여 REAL / FAKE 판정

2-1. Logistic Regression의 파라미터

HOW · 코드 근거

09_baseline_models.ipynb · cell 18 발췌

```
lr_model = LogisticRegression( max_iter=3000,
class_weight="balanced",
random_state=RANDOM_STATE, ) lr_model.fit(
X_train_scaled, y_train )
```

설정	역할
max_iter = 3000	최적화 반복 횟수 상한
class_weight = balanced	적은 REAL 표본의 손실 비중 보정
random_state = 42	재현성 설정
C = 1.0	코드에서 변경하지 않은 기본 규제값

실험에서 고정 설정, 별도 하이퍼파라미터 최적화 결과 X → 실제 튜닝 결과 급격한 변화를 보이진 않음

2-1. Logistic Regression의 결과

WHAT · 해석

Track 지표	Validation	Test
ROC-AUC	0.9621	0.9176
EER	0.1075	0.1385
Balanced Acc.	0.8925	0.8615
Macro-F1	0.7662	0.7523

평가 결과

0.6059

Validation에서 선택한 Track threshold

- Test REAL 오탐: 8 / 45곡
- Test FAKE 누락: 49 / 494곡
- Test Segment AUC 0.8776
- Track 평균 후 AUC 0.9176

해석 : TEST 결과 정확도 좋은편은 아님. 비선형 모델에 대해서 추가 개선 가능성 확인해볼 필요 있음.

2-2. RBF-SVM

WHY · HOW

모델 소개와 선정 이유

샘플 사이의 유사도를 이용해
곡선 형태의 경계를 학습

동일 266-D 입력에서
선형 LR의 한계를 보완하는지 검증

RBF 유사도

표준화한 특징 벡터의 거리를 계산

유클리디안 거리 : $K(x, s) = \exp(-\gamma \|x-s\|^2)$

가까운 샘플: 유사도 큼
먼 샘플: 유사도 작음

Support Vector

경계 형성에 중요한 학습 샘플

유사도의 가중합

decision score

확률 변환

predict_proba의 FAKE 점수

2-2. RBF-SVM의 파라미터

HOW · 코드 근거

09_baseline_models.ipynb · cell 25 발췌

```
svm_model = SVC( kernel="rbf", C=1.0,  
gamma="scale", class_weight="balanced",  
probability=True,  
random_state=RANDOM_STATE,  
cache_size=2048, )
```

설정	직관적 의미
C = 1.0	훈련 오류와 경계 규제의 균형
gamma = scale	한 샘플이 영향을 주는 범위
balanced	클래스 불균형 보정
probability = True	확률형 FAKE 점수 산출
cache_size = 2048	커널 계산용 메모리 MB

C, gamma 고정 설정, Scaler는 Train에만 fit

2-2. RBF-SVM의 결과

WHAT · 해석

Track 지표	Validation	Test
ROC-AUC	0.9569	0.9644
EER	0.1075	0.1395
Balanced Acc.	0.8925	0.8696
Macro-F1	0.7662	0.7502

평가 결과

0.8785

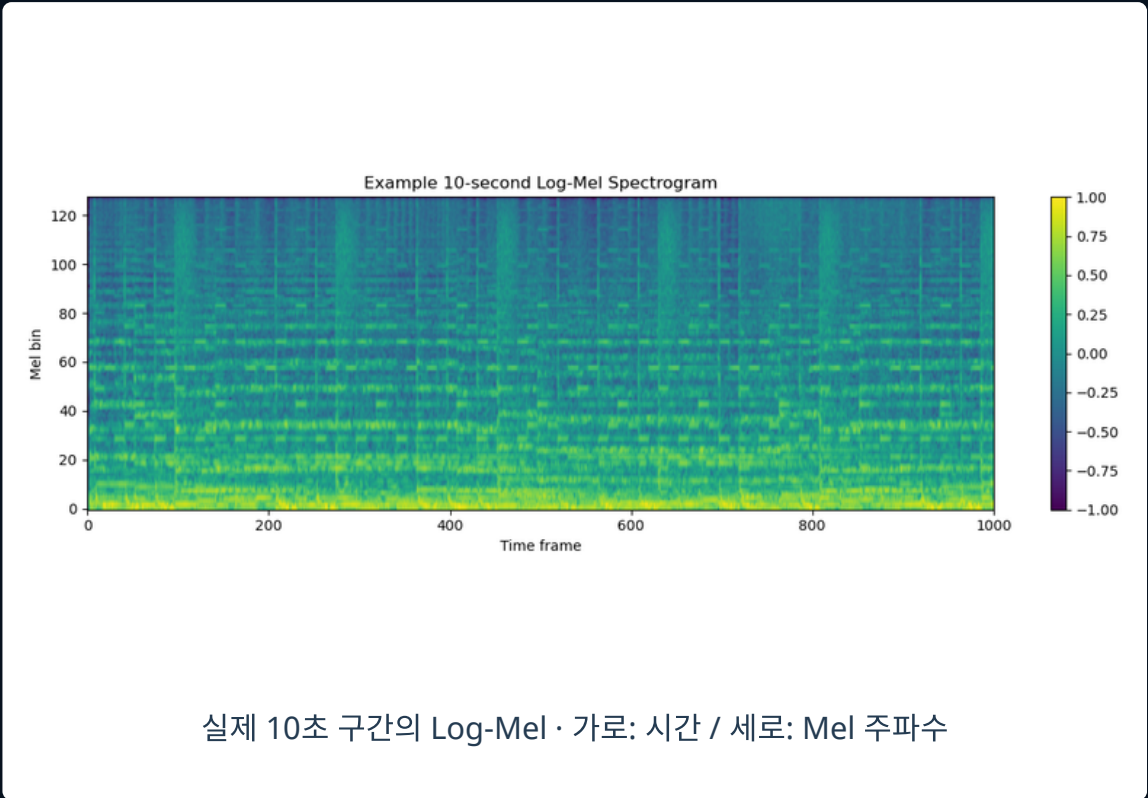
Validation에서 선택한 Track threshold

- Test REAL 오탐: 7 / 45곡
- Test FAKE 누락: 52 / 494곡
- Test Segment AUC 0.9262
- Track 평균 후 AUC 0.9644

해석 : TEST 결과 정확도 좋은편은 아님. 비선형 모델에 대해서 추가 개선 가능성 확인해볼 필요 있음.

2-3. Log-Mel CNN

WHY · HOW



통계 벡터에서 시간-주파수 지도로

- Handcrafted: 사람이 정한 평균·표준편차
- CNN: 국소적인 시간·주파수 패턴 학습

SETTING	VALUE	직관적 의미
sample rate	24,000 Hz	초당 24,000 sample
segment	10 sec	240,000 samples
n_fft	1,024	약 42.7 ms 창
hop_length	240	10 ms 간격
n_mels	128	128개 지각 주파수 band
fmax	12,000 Hz	Nyquist 상한

선택 이유: 시간 평균에서 사라질 수 있는 패턴을 CNN이 직접 학습하는지 검증

2-2. Log-mel CNN 파라미터

HOW · 구조 · 코드

Conv 1

1 → 16채널

Conv 2

16 → 32채널

Conv 3

32 → 64채널

Conv 4

64 → 128채널

13_logmel_cnn.ipynb · cells 33·40 발췌

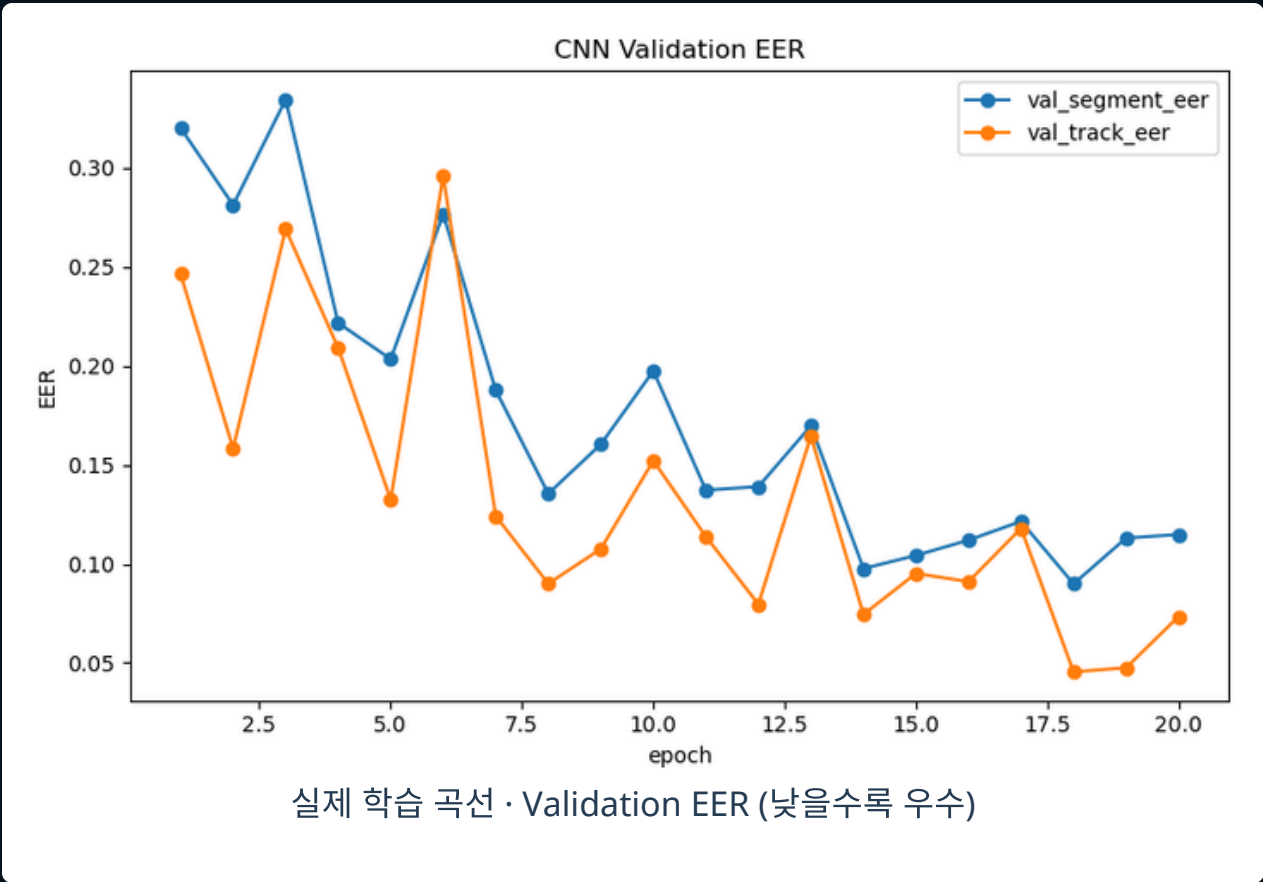
```
optimizer = torch.optim.AdamW(
model.parameters(), lr=1e-3, weight_decay=1e-4, )
MAX_EPOCHS = 20 PATIENCE = 4
```

ITEM	NOTEBOOK SETTING
Conv block	Conv 3×3 + BN + ReLU ×2, MaxPool 2×2
Parameters	294,321 trainable
Optimizer	AdamW · lr 1e-3 · weight decay 1e-4
Loss	BCEWithLogitsLoss · pos_weight = Train REAL / FAKE
Training	batch 32 · max 20 epoch · patience 4 · best epoch 18

구조·설정	역할
3×3 Conv·BN·ReLU ×2	각 block의 패턴 추출
MaxPool 2×2	시간·주파수 해상도 축소
AvgPool·Dropout 0.30·Linear	128차원 요약 후 logit 출력
batch 32 · 294,321 parameters	소규모 CNN 전체 학습
BCEWithLogitsLoss	pos_weight = 621 / 6,346

2-2. Log-mel CNN의 결과

WHAT · 해석



Track 지표	Validation	Test
ROC-AUC	0.9882	0.9772
EER	0.0455	0.1042
Balanced Acc.	0.9545	0.9111
Macro-F1	0.8762	0.8535
threshold	0.0935	0.0935

선택 가중치: epoch 18 · 학습 최대 20 epoch · Test REAL 오탐 6 / 45, FAKE 누락 22 / 494

Clean AUC는 SVM 보다 높았으며, 이 우위가 새 생성기와 압축에서도 유지되는지 뒤에서 검증하겠다.

2-4. MERT

WHY · HOW

음악 사전학습 표현

MERT는 음악을 이해하도록
사전학습된 Transformer encoder

- 작은 데이터에서 표현을 처음부터 학습하는 부담 감소
- encoder는 고정하고 LR만 학습
- 사전학습 표현의 탐지 유용성 검증

13개 level의 의미

level 0: 첫 Transformer block 입력

level 1-12: 각 block 출력

[유효 시간 frame, 768]

시간 평균 → level별 768-D

13개를 연결하지 않고 독립 후보로 비교

10초 waveform

24 kHz

Frozen MERT

level별 시간 평균

Scaler + LR

FAKE score

학습 가능한 부분은 분류기 · 768-D는 Handcrafted 266-D와 별개의 표현

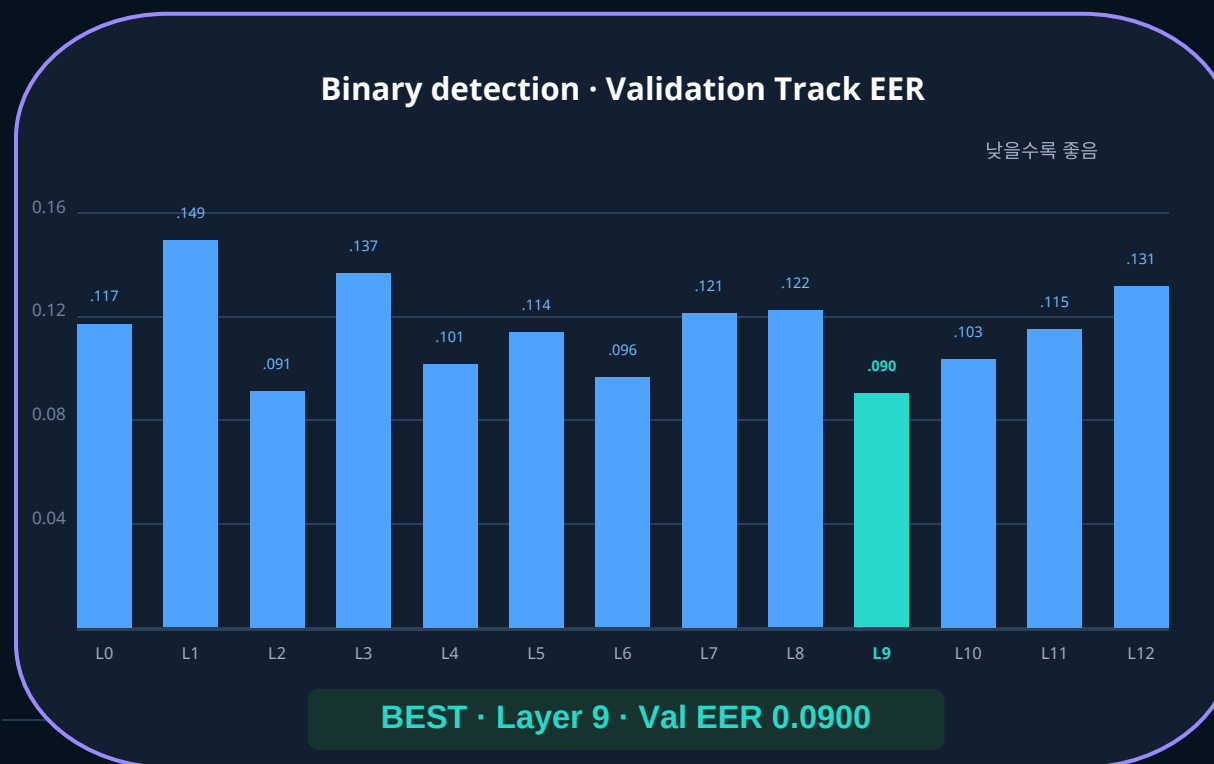
2-4. MERT 파라미터

HOW · 코드 · 선택 기준

17_mert_frozen_baseline_v2.ipynb · 핵심 발췌

```
for layer in range(13):
    X_train = cache[train_idx, layer, :]
    scaler = StandardScaler()
    X_train_z = scaler.fit_transform(X_train)
    clf = LogisticRegression(
        class_weight="balanced",
        max_iter=3000,
        random_state=RANDOM_STATE,
        solver="lbfgs",
    )
```

Validation 기준의 level 탐색



모델 m-a-p/MERT-v1-95M · 24 kHz · encoder frozen · C=1.0(기본값) · seed 42

2-4. MERT의 결과

WHAT · 해석

Track 지표	Validation	Test
ROC-AUC	0.9581	0.9872
EER	0.0900	0.0455
Balanced Acc.	0.9100	0.9363
Macro-F1	0.7896	0.8107
threshold	0.8365	0.8365

출력과 판정

level 9

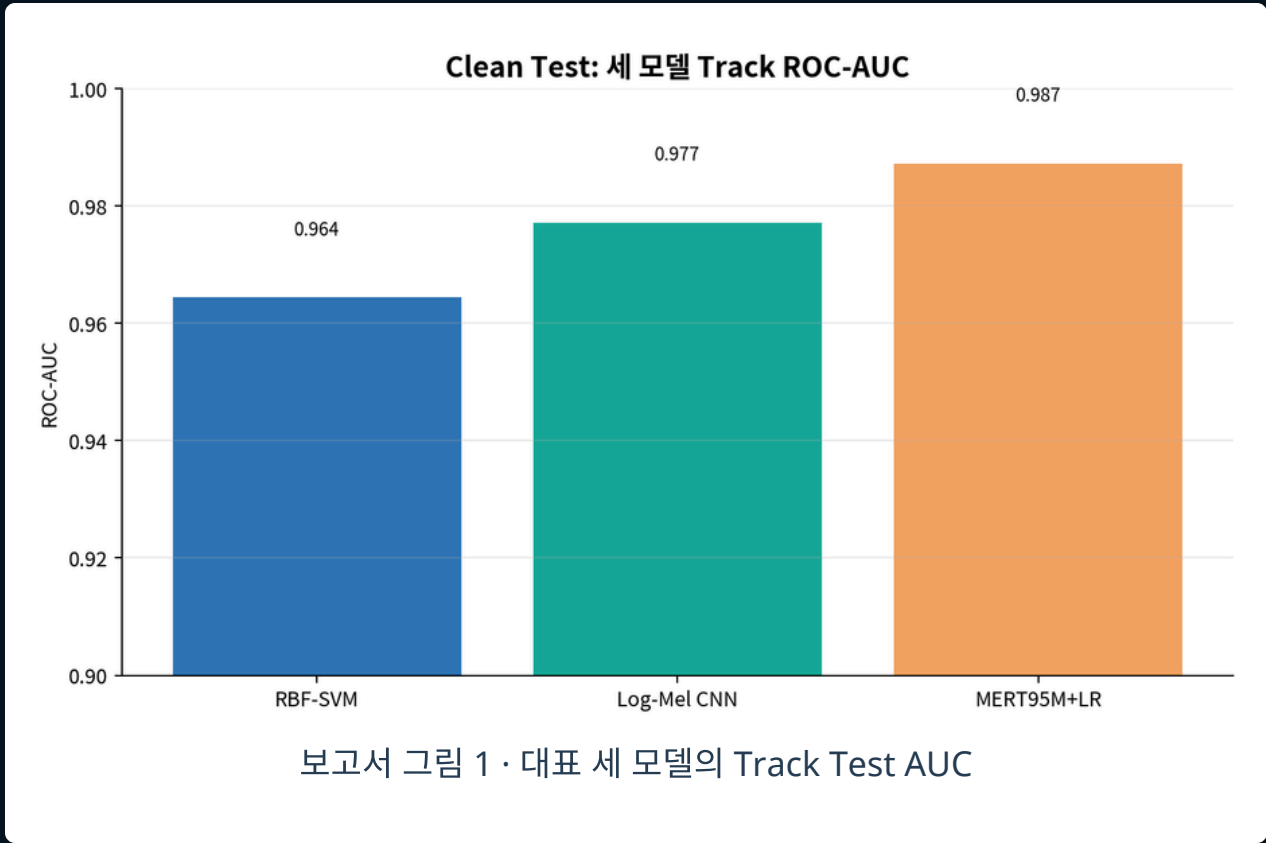
진위 탐지에 선택된 표현

- 구간별 FAKE 점수의 곡 단위 평균
- Test AUC·EER에서 가장 좋은 결과
- Test Macro-F1은 CNN보다 낮음

해석: 좋은 점수 순위와 좋은 고정 임계값 분류는 다른 문제 · 지표를 함께 비교

2-5. Test상 모델 비교

WHAT · 종합 비교



모델	AUC ↑	EER ↓	F1 ↑
LR	0.9176	0.1385	0.7523
SVM	0.9644	0.1395	0.7502
CNN	0.9772	0.1042	0.8535
MERT+LR	0.9872	0.0455	0.8107

AUC·EER: MERT 우위 / 고정 임계값 Macro-F1: CNN 우위

다음 질문: Clean 환경의 성능 순위가 새로운 생성기와 재압축에서도 유지되는가?

3. 후속 실험 설계

WHY · 후속 연구 설계

앞선 관찰	후속 질문	실험
성능 개선	1.새 생성기에도 적용 가능한가?	생성기 제외 학습
모델별 표현 차이	2.압축에 따라 신호가 달라지는가?	MP3 128 / 64 kbps 재인코딩
생성기별 탐지 난이도 차이	3.생성기 분류가 가능한가?	12개 생성기 식별
통제 후 식별 성능 유지	4.생성기 분류와 음향 특징은 무엇이 있는가?	PCA·UMAP / 장르·음향 특징
생성기별 원곡 범위 차이	5.장르 분류가 가능한가	Strict 균형 통제

후속 실험은 앞선 결과의 한계를 확인하는 과정 · 계획된 미래 연구와 수행 완료 실험을 구분

3-1. 새로운 생성기 일반화의 실험 설계

WHY · HOW

Why · 선정 배경

- 실제 환경에서는 새로운 생성기가 등장
- Clean 성능만으로 새 생성기 탐지 보장 불가
- MusicGen·Udio 두 조건으로 비교

How · 한 생성기씩 제외

- Train·Validation에서 해당 생성기의 FAKE 제거
- 남은 생성기로 모델을 다시 학습
- 남은 Validation으로 threshold 선택
- Test REAL 전체 + 제외 생성기 FAKE 평가

실험 A

MusicGen 제외

실험 B

Udio 제외

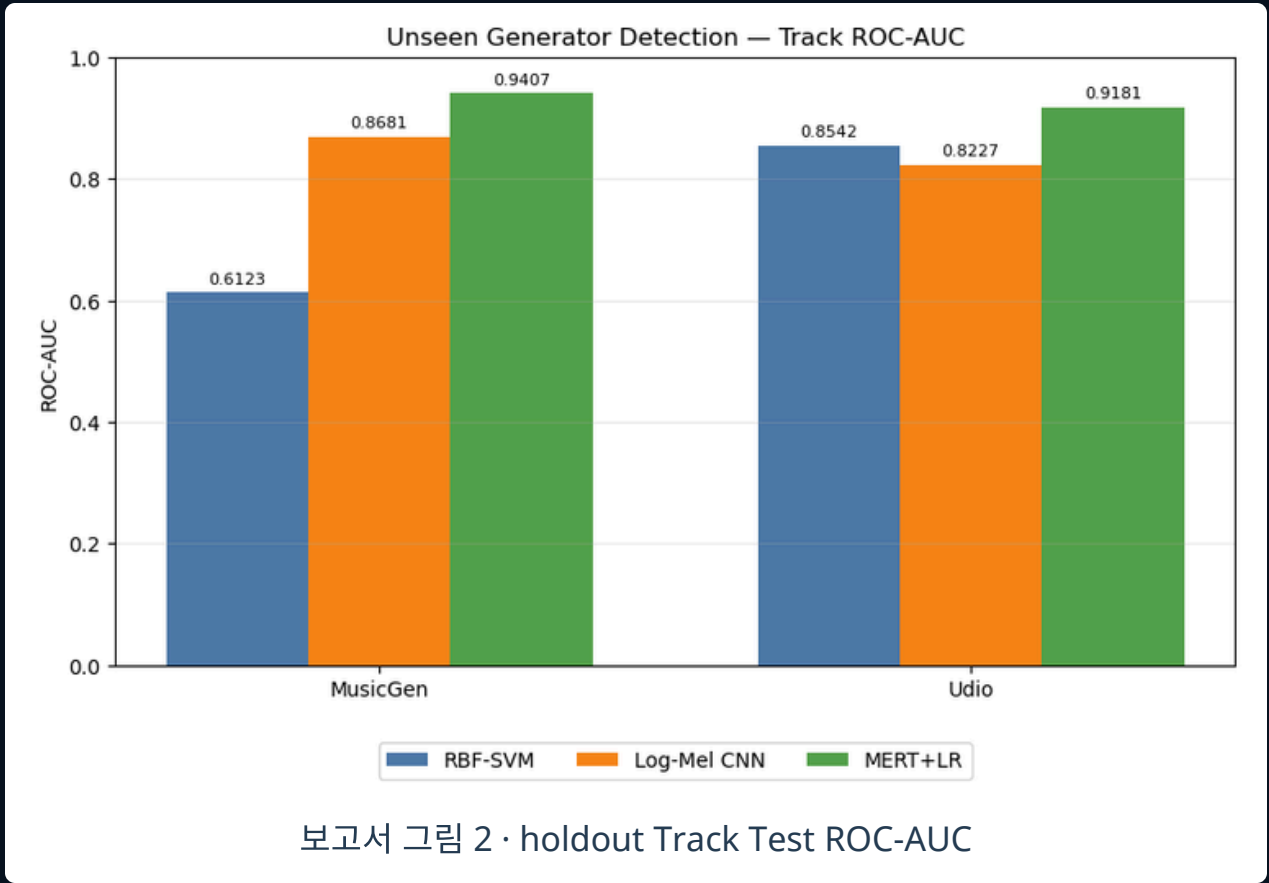
비교 모델

Handcrafted SVM · CNN · MERT

제외 대상은 Train뿐 아니라 Validation에서도 제거 · MERT 미지 생성기 실험은 미수행

3-1. 새로운 생성기 일반화의 결과

WHAT · 해석



Holdout	SVM	CNN	MERT
MusicGen	0.6123	0.8681	0.9407
Udio	0.8542	0.8227	0.9181

What · 모델별 차이

MusicGen: MERT가 가장 높음

Udio: CNN이 가장 낮음

SVM MusicGen AUC
in-domain 0.9951 → unseen 0.6123

학습에서 쉬웠던 생성기가 처음보는 조건에서도 쉬운 것은 아님을 알 수 있다.

3-2. MP3 압축 실험 설계

WHY · HOW

Why · 실제 유통 환경

업로드·스트리밍 과정에서
음악의 압축 조건 변화

모델이 사용한 음향 단서가
재압축 후에도 남는지 확인

How · 동일 음원 비교

- 같은 Test 539트랙을 재인코딩
- Original / MP3 128 / MP3 64 kbps
- 모델·scaler·Validation threshold 고정
- 같은 구간 계획으로 특징 재추출

고정 조건

내용·정답·평가 트랙

변경 조건

MP3 bitrate

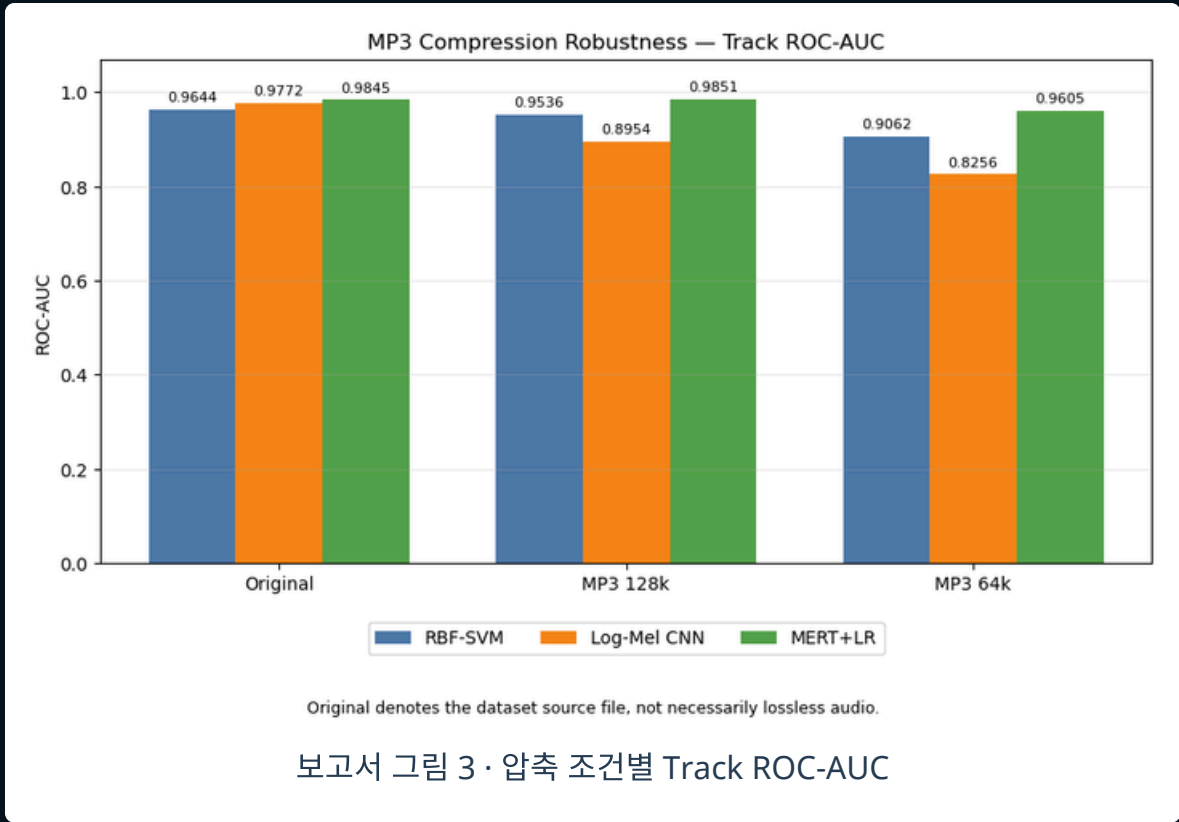
측정 결과

AUC 변화 · 오류율 변화

재학습 없이 입력 압축 조건만 변경 · Original도 원자료 조건이며 무손실 음원을 뜻하지 않음

3-2. MP3 압축 실험 결과

WHAT · 해석



CNN의 Udio AUC: 0.9671 → 0.7694 (64 kbps)

해석: CNN의 Clean 우위가 재압축에서 역전 · 민감한 음향 단서의 정체는 추가 검증 필요

조건	SVM	CNN	MERT
Original	0.9644	0.9772	0.9845
128 kbps	0.9536	0.8954	0.9851
64 kbps	0.9062	0.8256	0.9605
64 kbps 하락	-0.0583	-0.1516	-0.0240

3-3. 생성기 분류 실험 설계

WHY · HOW

Why · 탐지 신호의 확장

AI 여부를 넘어

어느 생성기가 만들었는지 확인

생성기별 음향 정보의
구분 가능성 탐색

How · 12개 클래스 분류

- REAL 제외 · FAKE 9,189구간 사용
- Handcrafted 266-D + RBF-SVM
- MERT 768-D + 다중 클래스 LR
- MERT level은 Validation Macro-F1로 선택

입력

FAKE 음원 표현

출력

12개 생성기별 점수

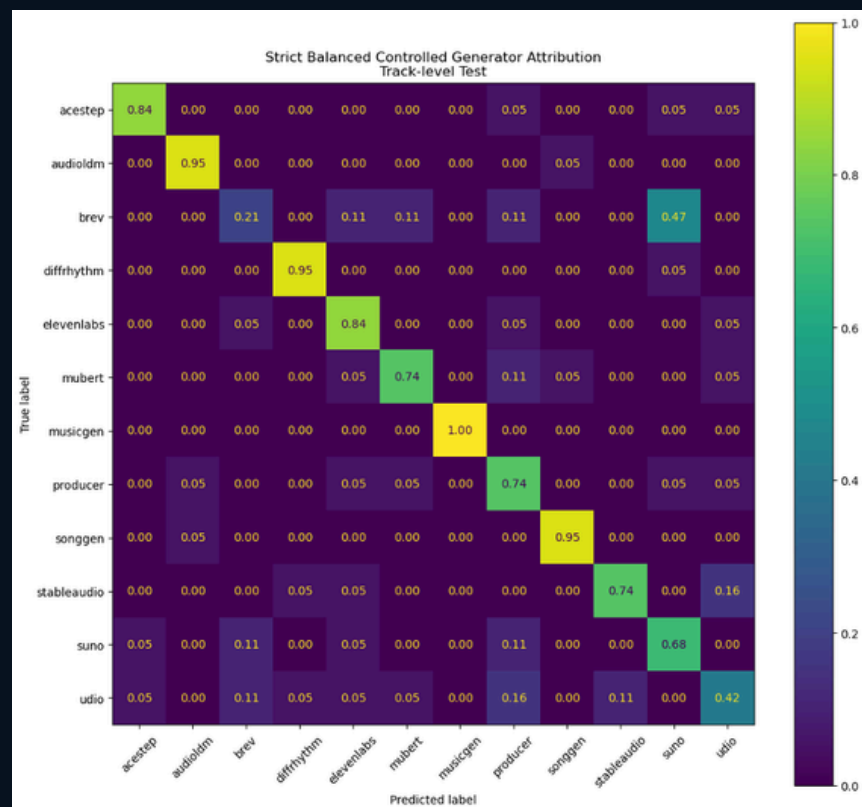
판정

최대 점수의 생성기

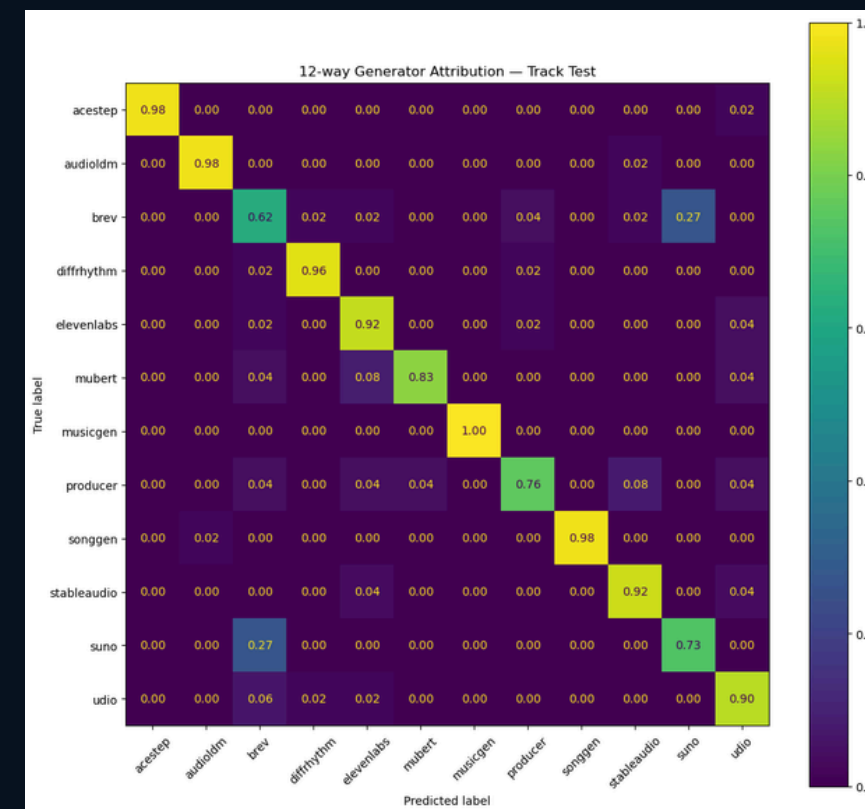
Binary: REAL / FAKE 구분 · Attribution: 알려진 12개 생성기 중 선택

3-3. 전체 데이터의 생성기 식별 결과

WHAT · 해석



실제 출력 · Handcrafted SVM의 Full Track 혼동행렬



실제 출력 · MERT의 Full Track 혼동행렬

비교적 잘 분류 하는 것처럼 보이나 혼동이 늘어나는 모습이 보임. 일반화된 결과로 판단하기 어려움 → 생성기 특징 확인 필요

3-4. 통제된 균형 데이터 실험 설계

WHY · HOW

Why · 구성 편향의 통제

- 생성기별 원곡 포함 범위 차이
- 생성기별 곡 수 149-300개
- 생성기별 장르 분포 차이

How · 동일 비교 집합

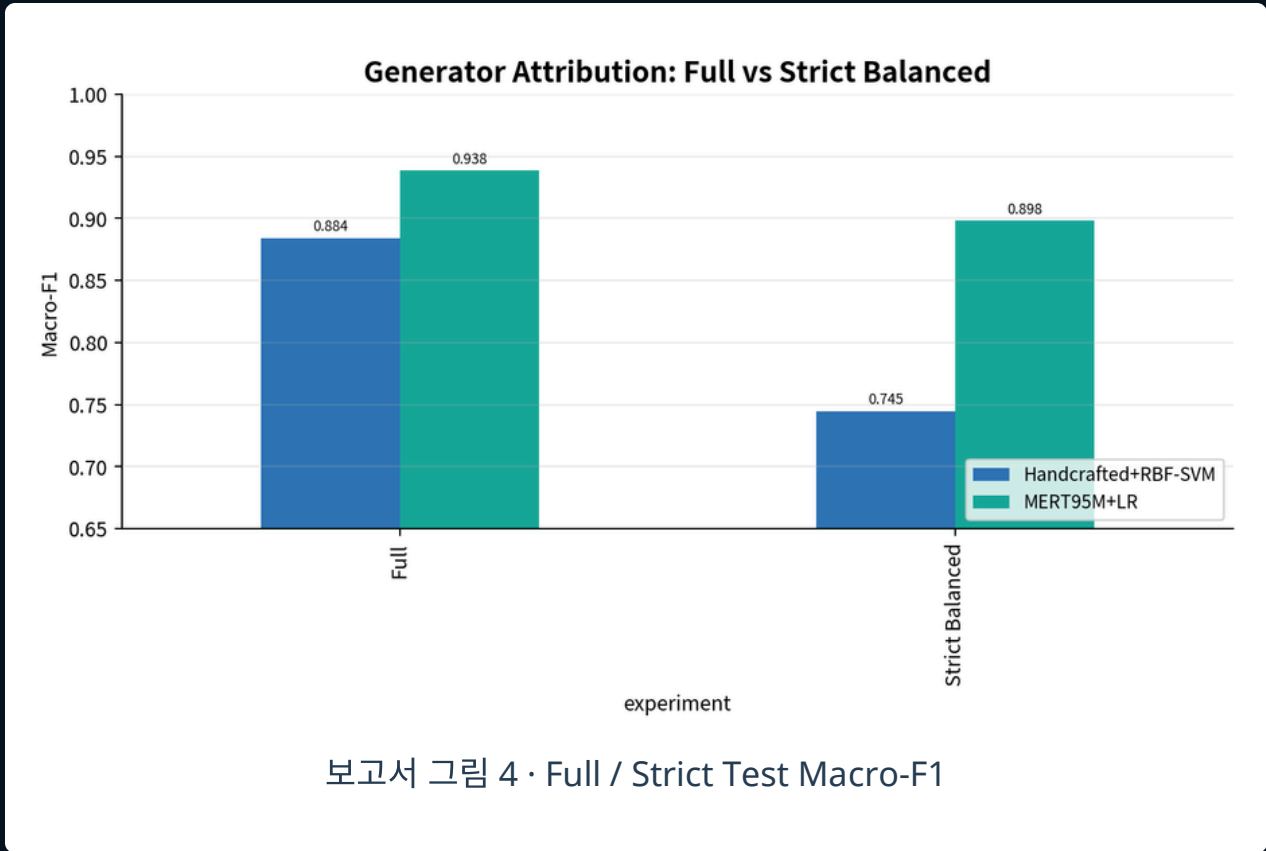
- 12개 생성기가 모두 있는 원곡만 선택
- 원곡 × 생성기 조합에서 정확히 1곡
- 원곡 단위 split 유지
- 동일 원곡·곡 수·장르 구성 확보

Strict split	원곡	생성기당 곡	전체 곡
Train	83	83	996
Validation	17	17	204
Test	19	19	228

119개 공통 원곡 × 12개 생성기 = 1,428트랙 · Test 생성기당 Electronic 7 / Pop 5 / Rock 7

3-4. 통제된 균형 데이터 실험 결과

WHAT · 해석



Strict Test	SVM	MERT+LR
Accuracy	0.7544	0.8991
Macro-F1	0.7447	0.8980
Top-3	0.9386	0.9912
Macro OVR AUC	0.9534	0.9920

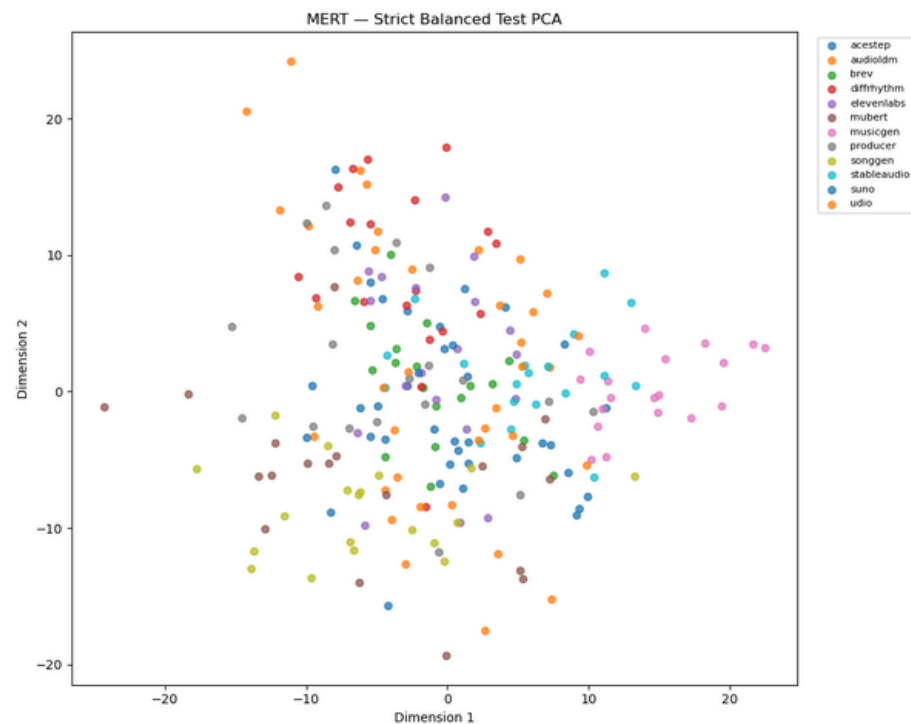
SVM Strict Validation F1: 0.7486

MERT Strict: level 10

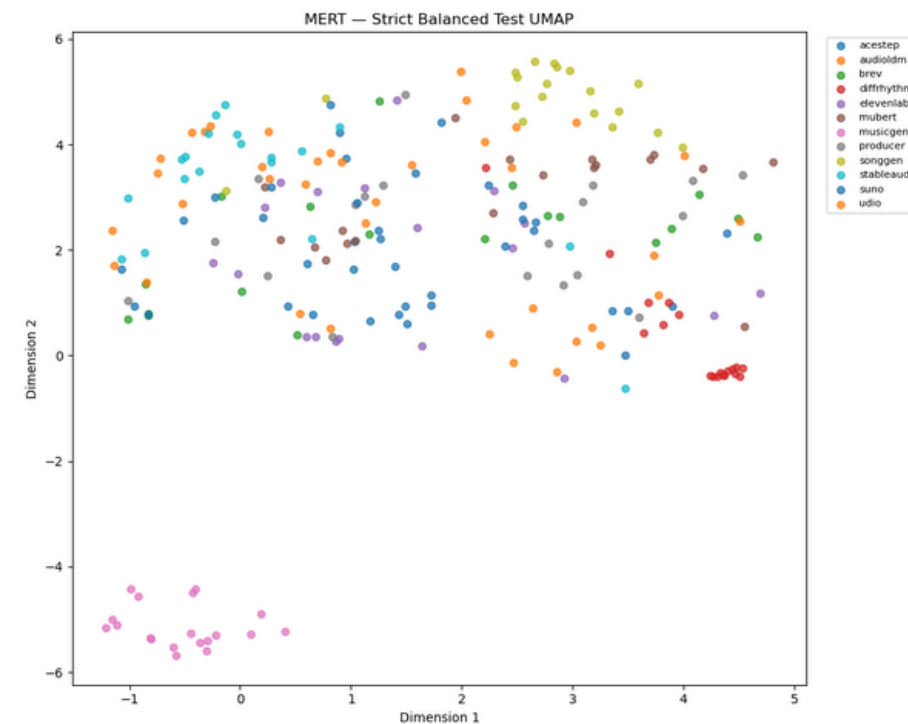
해석: 구성 통제 후에도 생성기 정보의 구분 가능성 유지 · 특정 생성 흔적의 인과적 증명은 아님

3-4. MERT 표현의 PCA·UMAP 시각화

WHY · HOW · WHAT



PCA · 큰 변동 방향을 2차원으로 요약

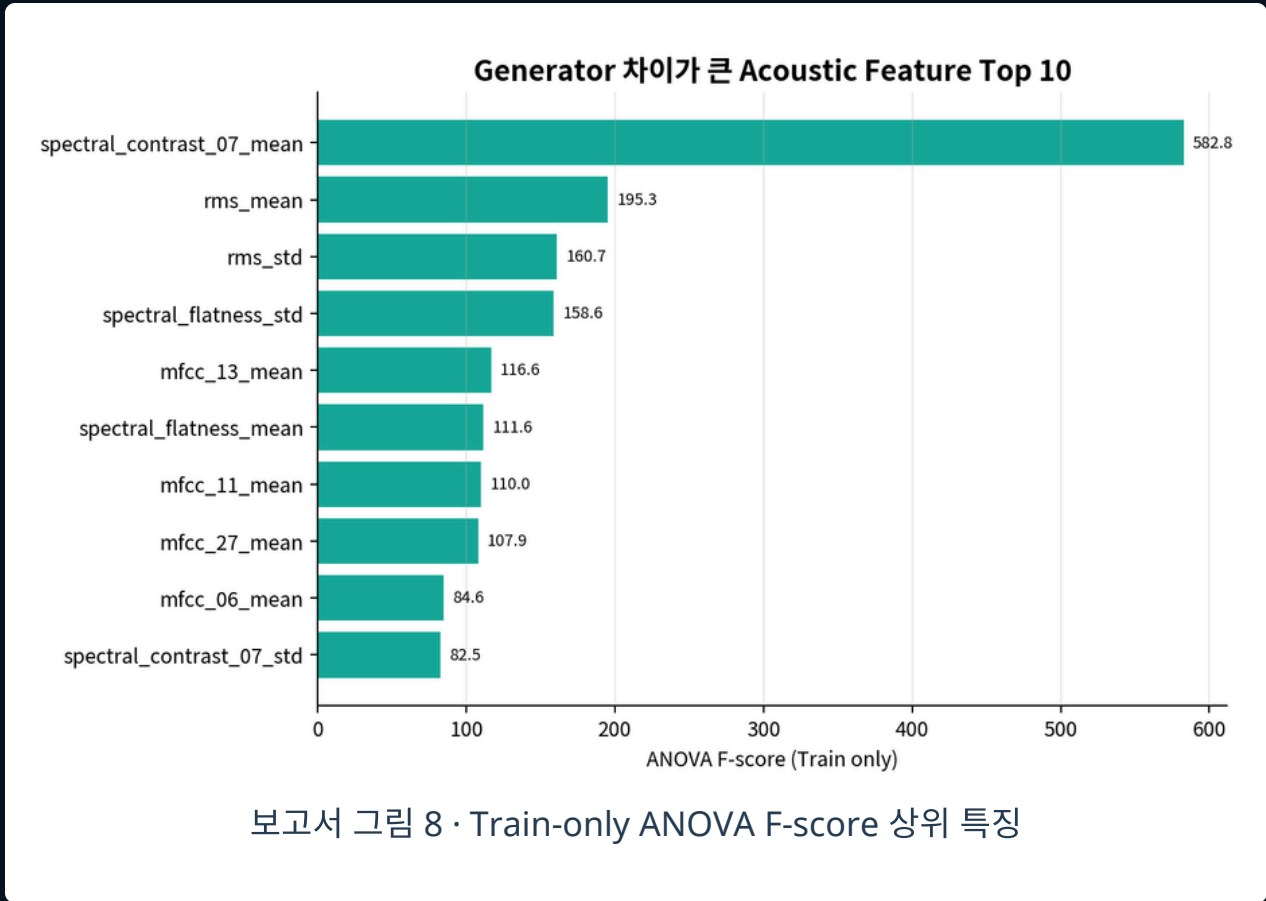


UMAP · 가까운 이웃 관계를 2차원에 배치

해석: 2D 군집 분리와 768-D 분류 성능은 다른 성질 · 선형 LR의 높은 성능과 공존 가능

3-4. 생성기 간 음향 특징 차이

WHY · HOW · WHAT



Why · 구분 단서 탐색

생성기 간 차이가 큰
음향 특징의 종류 확인

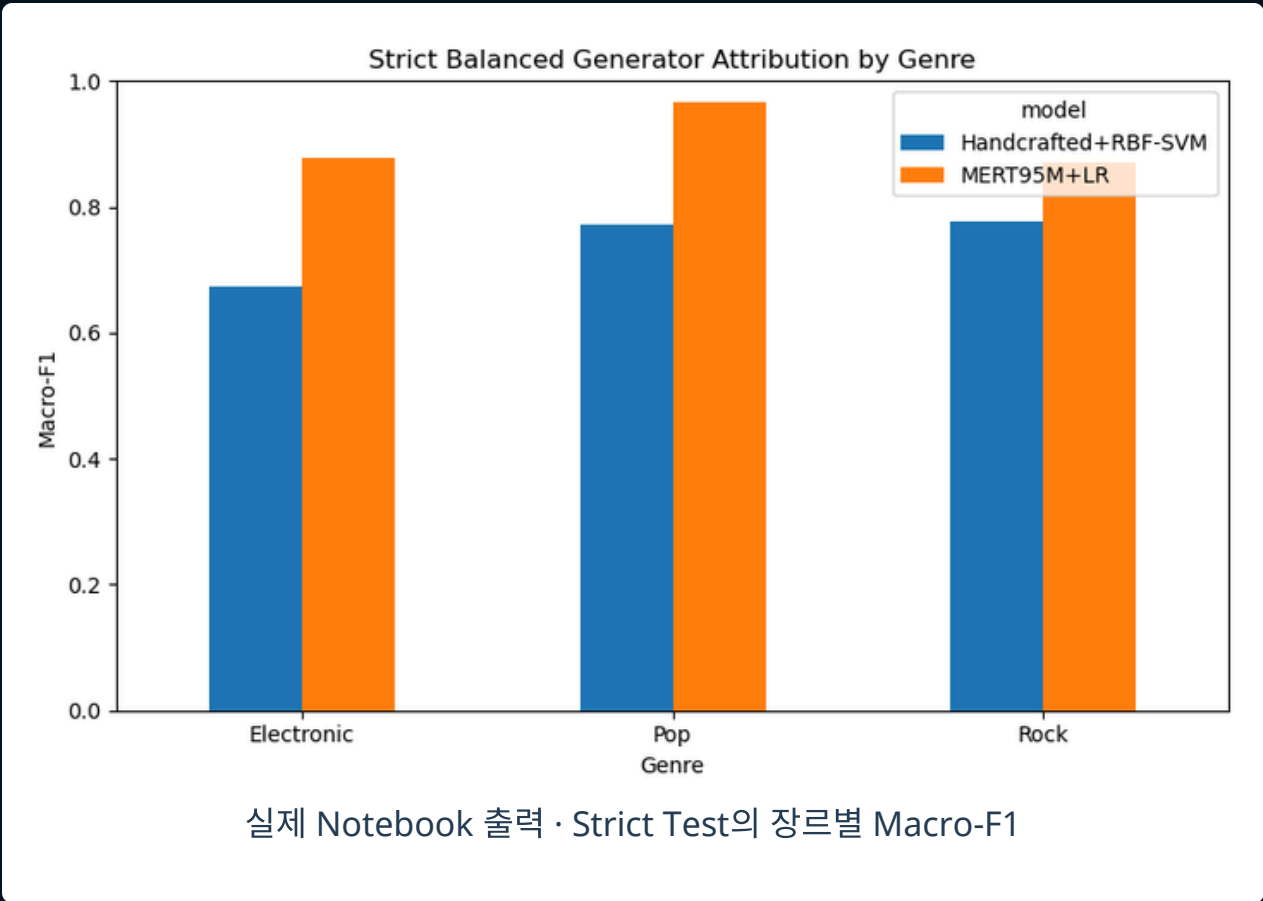
How · 각 특징의 생성기 간 평균 차이 비교

- Spectral contrast: 주파수 대역의 대비
- RMS: 신호 에너지 크기
- Flatness: 톤·잡음 성질
- MFCC: 음색 관련 요약

해석: ANOVA는 단변량 연관성 분석 · SVM/MERT의 직접적·인과적 중요도와 구분

3-5. 장르별 생성기 식별 성능

WHY · HOW · WHAT



Why · 장르 영향 확인

전체 평균에 가려진
장르별 난이도 차이 탐색

How · 같은 Strict Test를 장르별 집계

장르	MERT F1
Electronic	0.8767
Pop	0.9667
Rock	0.8700

해석: 세 장르에서 MERT 우위 · 장르별 표본 수가 작아 일반적 장르 난이도로 단정 불가

4. 핵심 결과와 연구 한계

확인된 결과

- **Clean**: MERT AUC 0.9872
- **재압축**: CNN의 하락 폭이 더 큼
- **새로운 생성기**: 생성기별 모델 우위 차이 존재
- **Strict 식별**: MERT F1 0.8980 유지

한계

- 296개 원곡 · 12개 생성기 · 3개 장르 표본적 한계
- 생성기별 분류시 데이터 통제를 했음에도 성능과 인과관계 설명이 부족
- 압축 환경에서의 성능 비교시 CNN의 성능 하락 요인 분석 필요

좋은 Clean 점수, 환경 변화 대응, 교란 통제 후 식별 성능을 별개의 근거로 평가

감사합니다